ELASTIC3rd Documentation

Release 7.0.2

ELASTIC3rd Developers

Oct 17, 2026

CONTENTS

1 Introduction **1**
- 1.1 Features . . . 1
- 1.2 Requirements . . . 1

2 Installation **3**
- 2.1 Requirements . . . 3
- 2.2 Install by pip . . . 3
- 2.3 Install from source . . . 3

3 Get Started **5**
- 3.1 Simplest way to run . . . 5
- 3.2 Inputs . . . 5
- 3.3 Calculation . . . 8
- 3.4 Post . . . 9
- 3.5 Outputs . . . 9

4 Energy and Stress Providers **13**
- 4.1 First-principles providers . . . 13
- 4.2 Machine-learning providers . . . 13
- 4.3 Choosing a provider for validation work . . . 14
- 4.4 Provider references . . . 14

5 Theory Background **17**
- 5.1 Strain-energy method . . . 17
- 5.2 Strain-stress method . . . 18
- 5.3 Numerical differentiation method . . . 18
- 5.4 Coefficient relation and higher-order effects . . . 18

6 Examples **19**
- 6.1 Help on the *elastic3rd* command . . . 19
- 6.2 Submit jobs . . . 20
- 6.3 Worked example: cubic Si with VASP, strain-energy, TOECs . . . 21
- 6.4 Worked example: tetragonal oxide with CASTEP, strain-stress, SOECs+TOECs . . . 22
- 6.5 Worked example: machine-learning force field (MACE) quick check . . . 22
- 6.6 Bundled example folders . . . 23
- 6.7 Post . . . 23

7 FAQ **27**

8 ELASTIC3rd API **29**

9 Citing 31

10 Author Information 33

11 Indices and tables 35

CHAPTER

ONE

INTRODUCTION

Elastic3rd is an open-source tool to calculate the higher-order elastic constants (HOECs) of materials from first-principles calculations or machine-learning force fields. It generates symmetry-adapted strain modes for a crystal, deforms the structure over a strain range, collects the energy or stress response from an external calculator, fits the response, and solves the resulting symmetry equations for the independent elastic constants of any requested order.

1.1 Features

- Second-, third- and fourth-order elastic constants for all 11 elastic crystal classes (triclinic, monoclinic, orthorhombic, tetragonal 1/2, rhombohedral 1/2, hexagonal 1/2, cubic 1/2); fifth-order is partially supported (cubic, tetragonal, orthorhombic, monoclinic, triclinic)
- Both the strain-energy method and the strain-stress method (higher orders require substantially fewer deformation modes with the strain-stress method)
- Two coefficient-recovery schemes: polynomial fitting and numerical differentiation (arbitrary accuracy stencils, 2/4/6/8)
- Post-processing that takes higher-order effects into account when recovering the constants from the fitted coefficients (M. Liao et al., Revisiting the third-order elastic constants of diamond: the higher-order effect, Diamond Relat. Mater. 117 (2021) 108490, doi:10.1016/j.diamond.2021.108490)
- First-principles providers: CASTEP (Materials Studio or standalone, Windows and Linux), VASP, a pymatgen-based VASP interface, ABACUS and Quantum ESPRESSO (energy only)
- Machine-learning providers executed in-process (no queue system needed): MACE, CHGNet, MatGL (M3GNet / TensorNet), SevenNet, MatterSim, ORB, GRACE, NequIP, DeepMD (DP), AlphaNet, TACE and Matris
- Command line interface (`elastic3rd run` / `elastic3rd post`) and a plain script-based API (`elastic3rd.elastic.elastic3`)
- Input files accepted as plain text, JSON or YAML; runs can be resumed with Continue mode
- Python 3.7+ on Windows and Linux

1.2 Requirements

python, numpy, scipy, matplotlib, pyyaml.

Optional dependencies for specific providers: pymatgen (VASP-pymatgen interface, ABACUS, and reading CIF/POSCAR structures for ML providers), ASE and the corresponding framework package for each machine-learning provider (see *Energy and Stress Providers*).

CHAPTER

TWO

INSTALLATION

2.1 Requirements

- Python 3.7 or newer
- numpy, scipy, matplotlib, pyyaml (installed automatically by pip)
- Optional, depending on the calculator you use:
  - pymatgen: required by the `vasp_pmg` interface, the ABACUS interface and by all machine-learning providers (to read CIF/POSCAR structures)
  - ASE plus the provider package itself (`mace-torch`, `chgnet`, `matgl`, `sevenn`, `mattersim`, `orb-models`, `grace-atomistic-models`, `nequip`, `deepmd-kit`, `alphanet`, `tace`, `matris`, ...) for machine-learning runs
  - abacuspy: required by the `abacus` interface

2.2 Install by pip

```
pip install elastic3rd
```

This also installs the `elastic3rd` command line entry point:

```
elastic3rd -h
```

2.3 Install from source

Clone or download the repository, enter the project root folder (the folder that contains `pyproject.toml`), and run:

```
pip install .
```

or, for a development install so that code edits take effect immediately:

```
pip install -e .
```

After installation you can check that everything works by importing the package:

```
python -c "import elastic3rd; print(elastic3rd.__version__)"
```

The machine-learning providers are imported lazily, so their heavy dependencies do not need to be installed unless you use them.

CHAPTER

THREE

GET STARTED

3.1 Simplest way to run

- Prepare the `INPUT.e3rd` file and the input files of your energy/stress provider (first-principles code or machine-learning force field)
- Enter the folder
- Run `elastic3rd run`

The whole workflow is orchestrated by a single function, `elastic3rd.elastic.elastic3(INPUT="INPUT.e3rd")`, which:

1. reads the input parameters and the lattice of the initial structure;
2. optionally geometry-optimizes the initial structure;
3. optionally adds a small atomic perturbation;
4. computes the reference energy/stress of the undeformed structure (`Mode0`);
5. applies each symmetry-adapted strain mode at each strain amplitude, deforming the lattice through the deformation gradient `F = strain2deformgrad(eta)`;
6. delegates the energy/stress calculation to the selected provider;
7. fits the response (polynomial fitting) or differentiates it numerically, then solves the symmetry equations for the independent elastic constants of every requested order.

The results (second-, third-, … order constants) are printed to the screen/log in GPa.

3.2 Inputs

The inputs for Elastic3rd can be divided into two parts. The first one is the `INPUT.e3rd` file, and the second is the input files for the energy/stress provider.

3.2.1 INPUT.e3rd

The file is a plain text list of `Key value` pairs (`#` starts a comment); JSON and YAML formats are also recognized from the file extension. Unknown keys raise an error. All keys and their defaults:

Parameter	Default	Comments
BaseName	Si	Name of the result folder. For CASTEP it must equal the stem of the .cell/.param files; for VASP it is any string; for the ML providers it is the stem of the structure file (BaseName.cif, BaseName.poscar, …)
CrystalType	C1	Elastic crystal class: cubic1/c1, cubic2/c2, hexagonal1/h1, hexagonal2/h2, rhombohedral1/r1, rhombohedral2/r2, tetragonal1/t1, tetragonal2/t2, orthorhombic/o, monoclinic/m, triclinic/n (case-insensitive, long or short names)
Ord	3	Order of the elastic constants (2, 3 or 4; 5 for c1/c2/t1/t2/o/m/n)
EnergyCode	CAST	Provider that computes energy/stress; Elastic3rd imports the module elastic3rd.energy.<EnergyCode> (case-insensitive), see EnergyCodes page
EnergyRun	1	1 = built-in run command (CASTEP: RunCASTEP.bat/sh -np NP BaseName; VASP: mpirun -np NP vasp_std). 0 = user defined, the command line is read from the energyrun file next to the provider module
FlagSE	E	E = strain-energy method, S = strain-stress method
CalMo	W	W = whole (all symmetry-adapted modes), S = user-specific modes read from the STRAINMODE file
GenSM	1	With CalMode=W: 1 = take built-in strain modes for this CrystalType/Ord/method (a STRAINMODE file is written out and can be reused with CalMode=S); 0 = generate modes programmatically
CoefMeth	fit	fit = polynomial fitting, num = numerical differentiation (num requires a symmetric strain list and STEPS = 5) num (not work now, please use command 'elastic3rd nd')
Acc	4	Accuracy of the numerical-differentiation stencil (2/4/6/8); ignored by the fitting method
MaxStrain	5.0	Maximum strain amplitude, in percent
MinStrain	-MaxS	Minimum strain amplitude, in percent
STEPS	5	Number of strain points per mode (at least 3)
StrainType	Uniform	Uniform = equidistant from MinStrain to MaxStrain; Cos… = cosine spacing. A STRAINAMP file overrides these four parameters when present (see below)
NP	1	Number of cores passed to the provider run command
Geom	0	Geometry optimization of the initial structure before deformation: 0 = none; 1 = full cell+ion relaxation (G); 2 = variable-cell shape relaxation then full relaxation (GS, G); 31/32 = same as 1/2 but stop after the optimization (useful to prepare a relaxed structure only)
StableEst	1	Treatment of each deformed structure: 1 = relax ions at fixed cell for every strain point, 0 = single-point calculation on the rigidly deformed lattice
AddPert	0	1 = add an atomic perturbation to the initial structure before the Mode0 calculation (helps symmetry-lowering relaxations converge)
PertMetho	random	Perturbation type: random or uniform (displacement ~2e-10)
Continue	0	1 = continue an interrupted run: existing Energy_*.txt / Stress_*.txt results are re-read instead of recalculating

A minimal example:

```
BaseName        Si
CrystalType     c1
Ord             3
EnergyCode      vasp
FlagSE          E
```

(continues on next page)

(continued from previous page)

```
CalMode         W
GenSM           1
CoefMethod      fit
MaxStrain       5.0
STEPS           5
NP              24
Geom            0
StableEst       1
Continue        0
```

3.2.2 Strain modes

For a given crystal class, elastic order and method, only a small number of symmetrically independent deformation modes is needed to recover all independent constants. With `CalMode W` these modes are either taken from the built-in library (`GenSM 1`, written to a `STRAINMODE` file) or generated so that the coefficient matrix of the requested order reaches full rank (`GenSM 0`).

Each mode is a Voigt 6-vector `xx yy zz yz xz xy` (engineering convention, in `elastic3rd/symmetry/COEFS` the built-in sets are stored per order and method), so with `CalMode S` the user can supply his/her own `STRAINMODE` file, e.g. the cubic set shipped in the `examples` folder:

```
0 0 0 2 0 0
0 0 0 0 2 0
0 0 0 0 0 2
1 0.5 0 0 0 0
1 -0.5 0 0 0 0
1 1 1 0 0 0
```

3.2.3 Strain amplitudes

The amplitudes applied to every mode are built from `MinStrain`, `MaxStrain`, `STEPS` and `StrainType`. If a file named `STRAINAMP` exists in the working folder, it is loaded instead (one vector of amplitudes in percent, whitespace separated) and overrides those four parameters. Numerical differentiation (`CoefMethod num`) additionally requires a symmetric list, e.g.:

```
-5 -2.5 0 2.5 5
```

3.2.4 Provider input files

What the provider expects in the working folder:

- CASTEP: `BaseName.cell` and `BaseName.param` (plus `RunCASTEP.bat` on Windows or `RunCASTEP.sh` on Linux in the current folder). Details on the CASTEP files.
- VASP: `INCAR`, `POSCAR`, `KPOINTS`, `POTCAR` (plus `mpirun` and a compiled `vasp_std` in `PATH`). Details on the VASP files. For the strain-stress method set `ISIF = 2` so that stress is printed in OUTCAR.
- ABACUS: `STRU`, `KPT`, `INPUT`.
- Quantum ESPRESSO: `BaseName.scf` (PWInput), results read from `BaseName.out` (energy only).
- ML providers: a structure file readable by pymatgen (`BaseName.cif`, `BaseName.poscar`, `BaseName.json`, `BaseName.yaml` or a plain `POSCAR`), optionally a `settings.yaml` to choose the model and optimizer.

3.3 Calculation

Run everything with:

```
elastic3rd run
```

or, equivalently, from a python script (handy on queue systems):

```
import elastic3rd.elastic
elastic3rd.elastic.elastic3()
```

which can be launched as `python runElastic3rd.py >> Result.txt`.

3.3.1 Using CASTEP

Note: The CASTEP interface is tested with the CASTEP module of Materials Studio, but it also works with standalone CASTEP. By default, the user should provide `RunCASTEP.bat` (Windows) or `RunCASTEP.sh` (Linux) in the current folder. Usually this file is located in `Materials_Studio_install_path\etc\CASTEP\bin`. Elastic3rd calls `RunCASTEP.bat -np NP BaseName` / `./RunCASTEP.sh -np NP BaseName`; the energy is read from `BaseName.geom` (or `BaseName.castep`), together with the relaxed structures, so ask for the appropriate `.geom`/stress output in your `.param` file.

3.3.2 Using VASP

For VASP, Elastic3rd runs `mpirun -np NP vasp_std` in each strain folder, so please ensure that `mpirun` and `vasp_std` can be found in your `PATH`. The energy is read from `OSZICAR` and the stress from `OUTCAR` (the `in kB` block; requires `ISIF = 2`). After a geometry optimization the `CONTCAR` is used as the new `POSCAR`. The interface rewrites `ISIF/NSW/IBRION` automatically depending on whether a relaxation or a single point is requested (see `StableEst` and `Geom`).

To avoid maintaining INCAR/KPOINTS by hand, the `vasp_pmg` provider (based on pymatgen) only requires a `POSCAR`; then the energy is read from `OUTCAR` and the stress from `vasprun.xml`.

3.3.3 Using machine-learning force fields

ML providers are executed in-process through ASE calculators: no run command and no queue system are needed, and Elastic3rd only requires the structure file. See *Energy and Stress Providers* for the list of supported models and the `settings.yaml` options.

3.3.4 Using other first-principles codes

If you want to use another code to calculate the energy or stress, create a python file named after the code (lowercase, e.g. `qe.py`) under `elastic3rd/energy/` that implements the provider API:

- `get_base_vec(BaseName)`: read the lattice vectors from the code's input file
- `write_base_vec(BaseName, BaseVec)`: write the (deformed) lattice vectors back
- `run(NP, BaseName)`: return the shell string that runs one calculation
- `get_energy(BaseName)`: read the total energy (eV) from the code's output
- `get_stress(BaseName)`: read the stress (GPa, Voigt) from the code's output — required for the strain-stress method
- `copy_files(BaseName, Path)` / `rem_files(BaseName)`: stage and clean working files
- `update_structure(BaseName, Path)`: write the relaxed structure back

- `update_input_for_geomopt(BaseName, FlagEG, KeepSym)`: switch the code's input between single point (`EO`), ion relaxation (`E`) and cell relaxation (`G`/`GS`)
- `add_perturbation(BaseName, tor, PertMethod)`: displace the atoms

Then select it in `INPUT.e3rd` with, e.g., `EnergyCode QE`.

3.4 Post

The SOECs, TOECs, … (up to the order actually solved) are printed in the log on the screen once the calculation finished.

To take higher-order effects into account (i.e. refit the collected energy/stress data without running any calculation), use the `elastic3rd post` command in the folder where the job was submitted:

```
elastic3rd post -n 4        # taking 4th-order effect into consideration
```

The higher-order-effect is described in M. Liao, et al., Revisiting the third-order elastic constants of diamond: the higher-order effect, Diamond Relat. Mater. 117 (2021) 108490, doi:10.1016/j.diamond.2021.108490.

For the full option list, run `elastic3rd post -h`. See *Examples* for a worked example.

Alternatively, the separate numerical-differentiation method (PRL 121, 216001), where each Cij is obtained component by component from dedicated strain-stress finite differences, is available as its own command:

```
elastic3rd nd -s 4          # SOECs+TOECs with 4% strain, all independent constants
```

It reads `BaseName/CrystalType/Ord/NP/Acc` from `INPUT.e3rd` (order 2 or 3, accuracy 2 or 4) and writes the results to the `BaseName-E3rd-ND` folder. Both stress-capable first-principles providers and the machine-learning providers can be used; for the latter the code forces a single-point evaluation of each rigidly deformed lattice (no relaxation). See *Examples* for the full option list.

3.5 Outputs

3.5.1 The structure of outputs

All the outputs are stored in the folder named `BaseName`, as set in the INPUT file. The structure of the results:

```
Si
├─INPUT
├─EEnergy.txt
├─Mode0
├─Mode1
│  ├─Energy_Mode.txt
│  ├─Strain-2.0
│  │  ├─Energy_Strain.txt
│  │  └─...
│  ├─Strain-4.0
│  └─...
├─Mode2
│  └─...
└─...
```

3.5.2 Output files

This section gives an explanation for each file generated by Elastic3rd.

File name	Comments
INPUT.e3rd	A backup of the INPUT file
EEnergy.txt	The energy (strain-energy method) or stress (strain-stress method) of every strain amplitude (rows) for each strain mode (columns)
Energy_Strain.txt	The response of a single strain point, energy in several units (eV, Hartree, kJ/mol, kcal/mol); Stress_Strain.txt analog for stress
Energy_Mode.txt	The energies of one strain mode as a function of strain; Stress_Mode.txt analog for stress
STRAINMODE	A copy of the strain modes actually used
STRAINAMP	A copy of the strain amplitudes actually used (when provided)

Folders	Comments
Mode0	Result of the undeformed (reference) structure
ModeN	Results of the Nth strain mode
StrainN	Within each mode, one folder per strain amplitude
Geom	(When `Geom` is set) the geometry-optimization run

Note: here N is an integer.

3.5.3 Units

- Energies are collected in eV per cell; `EEnergy.txt` and the log also report Hartree, kJ/mol and kcal/mol.
- Stresses are read/converted to GPa.
- The reported elastic constants (C2, C3, ...) are in GPa.

3.5.4 Log file

By default, the log is printed to the screen, and it can be saved by re-direction (`elastic3rd run >> Result.txt`). It can be divided into five parts.

- Part1: the logo and citation part

```
+============================================================+
| EEEEE L       A   SSSSS TTTTT IIIII CCCCC 33333 RRRRR DDD   |
| E     L      A A  S       T     I   C         3 R   R D  D  |
| EEEE  L     A   A SSSSS   T     I   C      3333 RRRRR D   D |
| E     L     AAAAA     S   T     I   C         3 R  R  D  D  |
| EEEEE LLLLL A   A SSSSS   T   IIIII CCCCC 33333 R   R DDD   |
+------------------------------------------------------------+
|             Version: ELASTIC3PY  version 7.0.2             |
|                Date: 2026-10-15                            |
|              Author: Liao Mingqing                         |
|              E_mail: liaomq1900127@163.com                 |
|     FGMS @ Harbin Institute of Technology(HIT)             |
|    URL:https://github.com/hitliaomq/ELASTIC3rd             |
+============================================================+
```

- Part2: the summary of the input parameters
- Part3: the result of the initial (and, if requested, optimized) structure:

```
====================Crystal Structure====================
BaseVec:
     5.4647128853     0.0000000210     -0.0000000454
     -0.0000000000     5.4647128382     -0.0000000222
     0.0000000000     0.0000000000     5.4647129020
Lattice Parameter:
    a         b         c      alpha   beta    gamma
5.464713  5.464713  5.464713  90.000  90.000  90.000
Volume : 163.193194972

Energy of undeformed structure:
       eV             Hartree            kJ/mol            kcal/mol
-858.6574758785  -31.5550611783  -82847.8330874176  -19801.1085892908
```

- Part4: the results of each strain of each strain mode

```
==================Deformed Crystal=======================
----------------------------------------------------------
Start calculating Mode 1
Start calculating Strain -5.0 in Mode 1
BaseVec:
     ...
Energy:
       eV             Hartree            kJ/mol            kcal/mol
-858.0935122952  -31.5343359114  -82793.4188860286  -19788.1032822074
End of Strain -5.0 in Mode 1

                       ...
```

- Part5: the post-processing part, i.e. the symmetry equations and the elastic constants:

```
==================Post Processing=========================
[11 12 44]
[153.05111127  65.0171889   73.22518077]
[111 112 123 144 155 456]
[-703.9889652  -435.4462948   -88.79485798   71.7458725  -257.35598339
  -51.04707056]
=========================!!!END!!!==========================
```

The first index list/line pair gives the second-order constants (here cubic: C11, C12, C44, in GPa), the second one the third-order constants.

CHAPTER

FOUR

ENERGY AND STRESS PROVIDERS

Elastic3rd does not compute energies or stresses itself. The value of `EnergyCode` in `INPUT.e3rd` selects a provider module `elastic3rd.energy.<EnergyCode>` (case-insensitive). Providers fall into two families:

- **First-principles codes**, launched as external shell commands per strain folder;
- **Machine-learning force fields**, executed in-process through ASE calculators (listed in `elastic3rd.elastic.RUNDIRECTCODE`), which need no run command and no queue system.

4.1 First-principles providers

EnergyCode	E/S	Files in the working folder	Notes
`vasp`	E + S	`INCAR`, `POSCAR`, `KPOINTS`, `POTCAR`	Energy from `OSZICAR`, stress from `OUTCAR` (requires `ISIF = 2`); runs `mpirun -np NP vasp_std`
`vasp_pm`	E + S	`POSCAR`	pymatgen-based; energy from `OUTCAR`, stress from `vasprun.xml`
`castep`	E + S	`BaseName.cell`, `BaseName.param`, `RunCASTEP.bat/sh`	Results read from `BaseName.geom`, else `BaseName.castep`; VCA supported
`castep_`	E + S	same as `castep`	older CASTEP variant
`abacus`	E + S	`STRU`, `KPT`, `INPUT`	via `abacuspy`; results from `abacus.json`; runs `mpirun -np NP abacus`
`qe`	E only	`BaseName.scf`, reads	Energy from `BaseName.out`; strain-energy method only; interface not yet tested

4.2 Machine-learning providers

For ML providers, place a structure file readable by pymatgen next to `INPUT.e3rd`: `BaseName.cif`, `BaseName.poscar`, `BaseName.json`, `BaseName.yaml` or a plain `POSCAR`. The provider relaxes the structure (or runs a single point) with the ASE calculator of the force field, then writes the `ENERGY` (eV) and `STRESS` (GPa) files that Elastic3rd reads.

Energy-Code	Package to install	Default model
mace	mace-torch	mace-matpes-pbe-0
chgnet	chgnet	CHGNet pretrained
matgl	matgl	TensorNet-MatPES-PBE-v2025.1-PES (also serves M3GNet models, e.g. M3GNet-MP-2021.2.8-PES)
sevenn	sevenn	7net-mf-ompa
mattersim	mattersim	MatterSim-v1.0.0-1M
orb	orb-models	orb-v3-conservative-inf-omat
grace	grace-atomistic-m	GRACE-2L-OMAT-large-ft-AM
nequip	nequip	compiled model under ~/.cache/nequip/
dpmd	deepmd-kit	frozen model under ~/.cache/deepmd/
alphanet	alphanet	checkpoint under ~/.cache/alphanet/
tace	tace	TACE-v1-OAM-M
matris	matris	matris_10m_oam

The model, device and optimizer are configured in an optional `settings.yaml` placed in the working folder, e.g.:

```
model: M3GNet-MP-2021.2.8-PES
device: cpu
optimizer: FIRE
ase_filter: ExpCellFilter
steps: 400
```

Common keys: `model`, `model_path/weights_path` (for locally trained models such as NequIP/DP/AlphaNet), `device` (cpu/cuda), `optimizer` (BFGS/FIRE/LBFGS/FIRE2), `ase_filter` (ExpCellFilter/FrechetCellFilter/StrainFilter/UnitCellFilter), `steps`, `fmax`, `dtype/precision` (sevenn/tace). Elastic3rd itself maintains the `flageg` and `constrain_symmetry` keys: `flageg` encodes what to do at each step (`EO` single point, `E` relax ions at fixed cell, `G` relax cell and ions, `C` cell only), matching the `Geom` and `StableEst` input parameters.

Energy and stress are taken from the final relaxed configuration (`get_potential_energy`, `get_stress`). Because the calculators run in-process, the `EnergyRun/NP` input values are ignored for this family.

4.3 Choosing a provider for validation work

ML force fields make full HOEC pipelines (many strain points x several modes) very cheap, which is convenient for testing an input setup before spending CPU-time on a first-principles production run. Note, however, that the elastic constants inherit the accuracy of the underlying model; report final values from the intended electronic structure method.

4.4 Provider references

When a calculation uses one of the providers, the corresponding publications should be cited.

First-principles codes:

- `vasp`: G. Kresse, J. Furthmuller, Efficient iterative schemes for ab initio total-energy calculations using a plane-wave basis set, Phys. Rev. B 54 (1996) 11169, doi:10.1103/PhysRevB.54.11169.
- `castep`, `castep_ac`: S. J. Clark et al., First principles methods using CASTEP, Z. Kristallogr. - Cryst. Mater. 220 (2005) 567, doi:10.1524/zkri.220.5.567.65075.
- `abacus`: K. Zhou et al., ABACUS: An electronic structure analysis package for the AI era, J. Chem. Phys. 163 (2025) 192501, doi:10.1063/5.0297563.

- `qe`: P. Giannozzi et al., QUANTUM ESPRESSO: a modular and open-source software project for quantum simulations of materials, J. Phys.: Condens. Matter 21 (2009) 395502, doi:10.1088/0953-8984/21/39/395502.

Machine-learning models and libraries:

- `mace`: I. Batatia, D. P. Kovacs, C. Simm, C. Ortner, G. Csanyi, MACE: Higher order equivariant message passing neural networks for fast and accurate force fields, Adv. Neural Inf. Process. Syst. 35 (2022) 11423, arXiv:2206.07697.
- `chgnet`: B. Deng et al., CHGNet as a pretrained universal neural network potential for charge-informed atomistic modelling, Nat. Mach. Intell. 5 (2023) 1031, doi:10.1038/s42256-023-00716-3.
- `matgl`: C. Chen, S. P. Ong, A universal graph deep learning interatomic potential for the periodic table, Nat. Comput. Sci. 2 (2022) 718, doi:10.1038/s43588-022-00349-3 (M3GNet); H. Simeon, G. de Fabritiis, TensorNet: Cartesian tensor representations for efficient learning of molecular potentials, Adv. Neural Inf. Process. Syst. 36 (2023), arXiv:2306.06482 (TensorNet); Y.-C. Ko et al., Materials Graph Library (MatGL), an open-source graph deep learning library for materials science and chemistry, npj Comput. Mater. 11 (2025), doi:10.1038/s41524-025-01742-y (MatGL library).
- `sevenn`: C. Park, K. Kim, Y. Hwang, I. S. Han, Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations, J. Chem. Theory Comput. 20 (2024) 4857, doi:10.1021/acs.jctc.4c00190.
- `mattersim`: L. Yang et al., MatterSim: A deep learning atomistic model across elements, temperatures and pressures, arXiv:2405.04967 (2024) (preprint).
- `orb`: W. Neumann et al. (Orbital Materials), Orb: A fast, scalable neural network potential, arXiv:2410.22570 (2024).
- `grace`: P. Bochkarev, Y. Lysogorskiy, R. Drautz, Graph atomic cluster expansion for semilocal interactions beyond equivariant message passing, Phys. Rev. X 14 (2024) 021036, doi:10.1103/PhysRevX.14.021036; Y. Lysogorskiy, P. Bochkarev, R. Drautz, Graph atomic cluster expansion for foundational machine learning interatomic potentials, arXiv:2508.17936 (2025) (preprint).
- `nequip`: S. Batzner et al., E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials, Nat. Commun. 13 (2022) 2453, doi:10.1038/s41467-022-29939-5.
- `dpmd`: D. Zhang, et al., A graph neural network for the era of large atomistic models, Npj Comput. Mater. 12 (2026) 276. doi: 10.1038/s41524-026-02146-2.
- `alphanet`: D. Yin et al., AlphaNet: Scaling up local-frame-based neural network interatomic potentials, npj Comput. Mater. 11 (2025) 332, arXiv:2501.07155.
- `tace`: Z. Xu, W. Xie, H. Xie, P. Hu, Towards universal property prediction in Cartesian space: TACE is all you need, arXiv:2509.14961 (2025) (preprint).
- `matris`: Y. Zhou, et al., MatRIS: toward reliable and efficient pretrained machine learning interatomic potentials, (2026). arXiv: 2603.02002.

CHAPTER

FIVE

THEORY BACKGROUND

5.1 Strain-energy method

The strain-energy method to calculate TOECs is based on the continuum elasticity theory. The base equation is as follows (note: the Einstein summation notation is used in the following equations):

$$E = E_0 + \frac{1}{2!}V_0 C_{ijkl}\eta_{ij}\eta_{kl} + \frac{1}{3!}V_0 C_{ijklmn}\eta_{ij}\eta_{kl}\eta_{mn} + ...$$

Where

- E is the energy of the deformed structure
- E_0 is the energy of the initial structure
- V_0 is the volume of the initial structure
- C_{ijkl} is SOECs
- C_{ijklmn} is TOECs
- η_{ij} is the Lagrangian strain

And the Lagrangian strain can be written as follows:

$$\eta_{ij} = \frac{1}{2}(F_{ki}F_{kj} - \delta_{ij})$$

The F is the deformation gradient, and can be expressed by the lattic vector of deformed structure(r') and intial structure (r).

$$r' = Fr$$

when using symmetrical strain, the deformation gradient can be expressed by the Lagrangian strain.

$$F = Q\sqrt{\lambda}Q^T$$
$$Y = 2\eta + I$$
$$Q = (y_1, y_2, y_3)$$
$$\lambda = diag(\lambda_1, \lambda_2, \lambda_3)$$

where y_i and λ_i are the eigenvector and eigenvalue of $\mathbf{Y}$, I is the identity matrix.

For each strain mode, we only adjust the amplitude. Hence, we can express the strain as a function of strain amplitude η, then we have

$$\frac{E - E_0}{V_0} = \frac{1}{2}A_2\eta^2 + \frac{1}{6}A_3\eta^3 + O(\eta^4)$$

Finally, we can choose different strain modes, then get different equations about TOECs and SOECs. By solving the equations, we can get the value of SOECs and TOECs.

5.2 Strain-stress method

Differentiating the energy expansion above with respect to strain gives the conjugate (second Piola-Kirchhoff) stress:

$$\sigma_{ij} = \frac{1}{V_0}\frac{\partial E}{\partial \eta_{ij}} = \frac{1}{2!}C_{ijkl}\eta_{kl} + \frac{1}{3!}C_{ijklmn}\eta_{kl}\eta_{mn} + ...$$

so fitting stress vs. strain yields constants of order N from a polynomial of degree $N-1$ instead of N. Elastic3rd converts the Cauchy stress printed by the first-principles code into the second Piola-Kirchhoff stress (`cauchy2pk`). In the strain-stress method, one deformation can generate more independent equation about elastic consatnts (up to 6 if the strain mode is chosen carefully), which means the strain-stress method (`FlagSE S`) needs less strain modes compared to strain-energy method (ref. Comput. Phys. Commun. 280 (2022) 108478).

5.3 Numerical differentiation method

The separate numerical-differentiation scheme of Phys. Rev. Lett. 121, 216001 (2018) (Cao *et al.*) treats every independent constant on its own, and is driven by the `elastic3rd nd` command (see *Examples*). Instead of fitting the response of a complete mode set with one polynomial, the constants are obtained directly as derivatives of the second Piola-Kirchhoff stress at zero strain:

$$C_{ijkl} = \left.\frac{\partial \sigma_{ij}}{\partial \eta_{kl}}\right|_{\eta=0}, \qquad C_{ijklmn} = \left.\frac{\partial^2 \sigma_{ij}}{\partial \eta_{kl}\,\partial \eta_{mn}}\right|_{\eta=0}.$$

For each constant, the program generates a dedicated set of strain modes acting only on the components that survive the permutation symmetry of that constant (Eq. 6-8 of the reference, implemented in `elastic3rd.tools.sep_nd`), so the requested derivative is read off the conjugate stress component (converted from the Cauchy stress by `cauchy2pk`) without solving any coupled system.

The derivatives are estimated with finite-difference stencils symmetric around $\eta = 0$. The accuracy parameter `Acc` sets the number of points: $\pm\varepsilon$ for `Acc 2`, $\pm\varepsilon/2$ and $\pm\varepsilon$ for `Acc 4`, and so on up to 8-point stencils. The stencil weights are generated for the actual strain list by solving the linear system that makes the formula exact for polynomials up to the stencil order (`elastic3rd.tools.nd_coef`); when energy rather than stress is differentiated, the vanishing first derivative at equilibrium is imposed as an extra condition, which reduces the number of points needed for the same accuracy.

Because no global polynomial model is assumed, each estimate uses the fewest possible strain points and applies to every crystal class without a fitted mode set; the price is that each constant needs its own calculations, and a bare central difference inherits the numerical noise of the stress. Elastic3rd therefore evaluates every constant at each requested strain amplitude, writes the individual estimates to `EC.json` and their average to `EC_ave.json`, and prints the averaged constants in GPa; the spread over amplitudes is a direct, assumption-free error estimate. The method requires stress output, so it works with `vasp`, `castep` and `abacus`.

5.4 Coefficient relation and higher-order effects

In both cases, the symmetry relations among the tensor components (Brugger for 2nd/3rd, Brendel for 4th) are encoded in `elastic3rd/symmetry`: each strain mode contributes a linear equation that relates the fitted coefficients to the independent constants, and the system is solved exactly when the mode set is complete, or by least squares otherwise.

Fully supported independent-constant tables (and built-in strain modes) exist for 2nd, 3rd and 4th order over all 11 elastic crystal classes.

Truncating the expansion at the order being fitted biases higher-order constants when the strain window is large (ref. https://doi.org/10.1016/j.diamond.2021.108490). The `elastic3rd post` command refits the stored data with an extra polynomial order and reports the constants after the higher-order terms are taken into account (`Flag_Ord` up to 9), together with the convergence of each constant versus the strain window.

CHAPTER

SIX

EXAMPLES

6.1 Help on the *elastic3rd* command

There are two commands to submit jobs or do post processing.

- *elastic3rd run* command

```
ELASTIC3RD version: 7.0.2
Copyright © FGMS@HIT & JUST

usage: elastic3rd run [-h] [-i INPUT]

optional arguments:
  -h, --help            show this help message and exit
  -i INPUT, --input INPUT
                        The input file for Elastic3rd. Default: 'INPUT.e3rd'
```

- *elastic3rd post* command

```
ELASTIC3RD version: 7.0.2
Copyright © FGMS@HIT & JUST

usage: elastic3rd post [-h] [-n N] [-i INPUT] [-sm STRAINMODE] [-fig FIG] [-v VOLUME] [-
↪e ENERGY]

optional arguments:
  -h, --help            show this help message and exit
  -n N, --n N           The order of elastic constant taken into consideration in the␣
↪post
                        process. Default: 3
  -i INPUT, --input INPUT
                        The input file for Elastic3rd. Default: 'INPUT.e3rd'
  -sm STRAINMODE, --strainmode STRAINMODE
                        The filename of strainmodes. Default: 'STRAINMODE'
  -fig FIG, --fig FIG   Show the fitting fig. Default: 0
  -v VOLUME, --volume VOLUME
                        The volume of initial structure (only needed for the strain-
↪energy
                        method without a structure file at hand). Default: derived from␣
↪the
                        structure
  -e ENERGY, --energy ENERGY
```

(continues on next page)

(continued from previous page)

```
                        The file of the energy/stress result. It will try to find it in␣
↪the
                        current folder or in the BaseName folder. Default: 'EEnergy.txt'
```

- *elastic3rd nd* command (separate numerical differentiation, PRL 121, 216001)

```
ELASTIC3RD version: 7.0.2
Copyright © FGMS@HIT & JUST

usage: elastic3rd nd [-h] [-i INPUT] [-n N] [-s STRAIN [STRAIN ...]] [-a ACC] [--np NP]

optional arguments:
  -h, --help            show this help message and exit
  -i INPUT, --input INPUT
                        The input file for Elastic3rd. Default: 'INPUT.e3rd'
  -n N, --n N           The order of elastic constants, 2 or 3. Default: the Ord in the␣
↪input file
  -s STRAIN [STRAIN ...], --strain STRAIN [STRAIN ...]
                        The strain amplitudes, in unit of %. Multiple values give a mean␣
↪over
                        them. Default: the MaxStrain in the input file
  -a ACC, --acc ACC     The accuracy of the differentiation, 2 or 4. Default: the Acc in␣
↪the
                        input file
  --np NP               The number of cores used in the energy/stress calculation.␣
↪Default:
                        the NP in the input file
```

This command runs the component-wise numerical-differentiation scheme on the strain-stress response: every independent Cij is obtained from its own dedicated set of strain modes (Eq. 6-8 of the PRL), instead of fitting all modes together. The results are written to the folder `BaseName-E3rd-ND` (`EC.json` keeps every estimate per strain amplitude, `EC_ave.json` the average), and the averaged constants (GPa) are printed when the run completes. It works with the first-principles providers that return stress (`vasp`, `castep`, `abacus`) and with the machine-learning providers: for the latter, `flageg` is forced to `E0`, i.e. the stress of each rigidly deformed lattice is evaluated in-process at a single point without any relaxation, exactly as required by the method.

6.2 Submit jobs

There are several examples in the examples folder.

For Si and C, both VASP and CASTEP input files are prepared in the folder, and for Mg, only CASTEP input file is prepared. For the ABACUS/ML setups used in the tests, look at the test folder.

The user can run these examples by following steps

1. For copy right, the author can't provide the RunCASTEP.bat or RunCASTEP.sh (for CASTEP) and POTCAR (for VASP) files. The user should provide these file in the current folder.
   - For CASTEP, the user should copy *RunCASTEP.bat* or *RunCASTEP.sh* in their Materils Studio installation folder.
   - For VASP, the user should provide the *POTCAR* file
2. Submit the job

There are several ways to submit the job.

- Using *elastic3rd run* command.
- Using the *runElastic3rd.py* (in *examples* folder) script by running *python runElastic3rd.py*. If the user want to save the log into file, please run *python runElastic3rd.py >> Result.txt*
- Submit jobs in queue system. If the user want to run Elastic3rd in queue system, the following is an example. Please note that it assume the python and vasp is exists in the *PATH*

```
#PBS -l nodes=1:ppn=24
#PBS -l walltime=48:00:00
#PBS -l pmem=8gb
#PBS -A open
#PBS -j oe

cd $PBS_O_WORKDIR

python runElastic3rd.py
```

6.3 Worked example: cubic Si with VASP, strain-energy, TOECs

Folder layout before running:

```
Si/
  INCAR  KPOINTS  POSCAR  POTCAR
  INPUT.e3rd
```

`INPUT.e3rd`:

```
BaseName        Si
CrystalType     c1
Ord             3
EnergyCode      vasp
FlagSE          E
CalMode         W
GenSM           1
CoefMethod      fit
MaxStrain       5.0
STEPS           5
NP              24
Geom            0
StableEst       1
Continue        0
```

Make sure `ISIF = 2` if you later switch to the stress method; for the energy method any `ISIF` giving the total energy is fine. Run:

```
elastic3rd run
```

The code creates the six built-in cubic strain modes (`STRAINMODE`), one folder per mode and strain under `Si/`, and finally prints the SOECs `[C11, C12, C44]` and TOECs `[C111, C112, C123, C144, C155, C456]` in GPa.

6.4 Worked example: tetragonal oxide with CASTEP, strain-stress, SOECs+TOECs

This is the `examples` folder setup (a commented reference `INPUT.e3rd` is shipped with the source):

```
BaseName        C
CalMode         W
GenSM           1
FlagSE          S
CoefMethod      Fit
Continue        1
CrystalType     T1
Ord             2
EnergyCode      castep
EnergyRun       1
MaxStrain       1.00
MinStrain       -1.00
STEPS           5
Acc             4
NP              1
```

Provide `C.cell`, `C.param` (request the `.geom` output with stresses) and `RunCASTEP.sh`/`RunCASTEP.bat`, then run `elastic3rd run`. With `FlagSE S` the stress response is fitted instead of the energy, which for low symmetry needs far fewer modes.

6.5 Worked example: machine-learning force field (MACE) quick check

Prepare the structure and settings:

```
Cu/
  Cu.cif
  INPUT.e3rd
  settings.yaml
```

`INPUT.e3rd`:

```
BaseName        Cu
CrystalType     c1
Ord             3
EnergyCode      mace
FlagSE          E
CalMode         W
GenSM           1
CoefMethod      fit
MaxStrain       5.0
STEPS           5
StableEst       1
```

`settings.yaml`:

```
model: mace-matpes-pbe-0
device: cuda
optimizer: FIRE
ase_filter: ExpCellFilter
steps: 400
```

Run `elastic3rd run` — the relaxations and single points are performed in-process by the MACE calculator, no queue submission. The same pattern works for `chgnet`, `matgl` (set e.g. `model: M3GNet-MP-2021.2.8-PES`), `sevenn`, `mattersim`, `orb`, `grace`, `tace`, `matris`, `nequip`, `dpmd` and `alphanet`.

6.6 Bundled example folders

In addition to the older Si/C/Mg setups, the examples folder ships three complete worked setups.

`vasp-Diamond-differentMethods` (cubic diamond, VASP, four methods side by side)
One input folder with four sub-folders, each a self-contained run of the same structure (`POSCAR`), `INCAR`/`KPOINTS` files (**Note:** `POTCAR` file is required to run the examples) and a user-provided `STRAINMODE` file (six modes for the cubic class: `1 0 0 0 0 0`, `1 1 0 0 0 0`, `1 0 0 2 0 0`, …):

- `Ord2`: second order only (`Ord 2`, `FlagSE S`, `MaxStrain 1`, 7 points)
- `SEM`: strain-energy method (`FlagSE E`, `Ord 3`, `MaxStrain 6`, 13 points)
- `SSM`: strain-stress method (`FlagSE S`, same strain grid as `SEM`)
- `ND`: the separate numerical-differentiation workflow, run with `elastic3rd nd` (`MaxStrain 4`)

All of them set `CrystalType c1`, `CalMode S` (built-in modes are replaced by the shipped `STRAINMODE`), `Geom 1` (the equilibrium structure is relaxed first) and `Continue 1`, so an interrupted run can simply be restarted. The comparison of the three recovery schemes on this data, including the higher-order effect on the TOECs of diamond, is the subject of M. Liao, Y. Liu, Y. Wang et al., Revisiting the third-order elastic constants of diamond: the higher-order effect, Diamond Relat. Mater. 117 (2021) 108490, doi:10.1016/j.diamond.2021.108490.

`ML-Diamond` (cubic diamond, all machine-learning providers)
Eleven sub-folders (`mace`, `matgl`, `sevenn`, `mattersim`, `orb`, `grace`, `nequip`, `dpa`, `alphanet`, `tace`, `matris`) with identical `INPUT.e3rd` files except for the `EnergyCode` key (the `dpa` folder uses the provider name `dpmd`). The setup mirrors the VASP strain-stress run above (`FlagSE S`, `CalMode S` with a shipped `STRAINMODE`, `CrystalType c1`, `Ord 3`, `MaxStrain 6`, 13 points): the energy/stress of every deformed lattice is computed in-process by the corresponding ASE calculator, so a full TOEC pipeline per model runs on a workstation. Drop a `settings.yaml` (model, device, optimizer) into a sub-folder to select the model; see *Energy and Stress Providers*.

`abacus-Mo` (molybdenum, ABACUS)
The `E3rd` sub-folder holds the Elastic3rd input (`INPUT.e3rd` with `EnergyCode abacus`, `BaseName Mo`, `CrystalType c1`, `Ord 3`, `FlagSE S`, `CalMode S`, `GenSM 0` with a user `STRAINMODE` file, symmetric grid from `MinStrain -5` to `MaxStrain 5`, 11 points, `Geom 1`), while the ABACUS inputs (`STRU`, `KPT`, `INPUT`) sit in the parent folder. The provider drives `abacuspy` and reads the results from `abacus.json`; see *Energy and Stress Providers* for the required files.

6.7 Post

There are several ways to do the post processing.

- Using *elastic3rd post* command

Assume that the current folder is the folder where the job has been submitted. When enter the current folder, the user can do the post process by *elastic3rd post* to get the SOECs and TOECs. For more options, please run *elastic3rd post -h*.

For test, the user can run elastic3rd post in the *example/Si/CASTEP* folder directly. It should print the following lines in the screen.

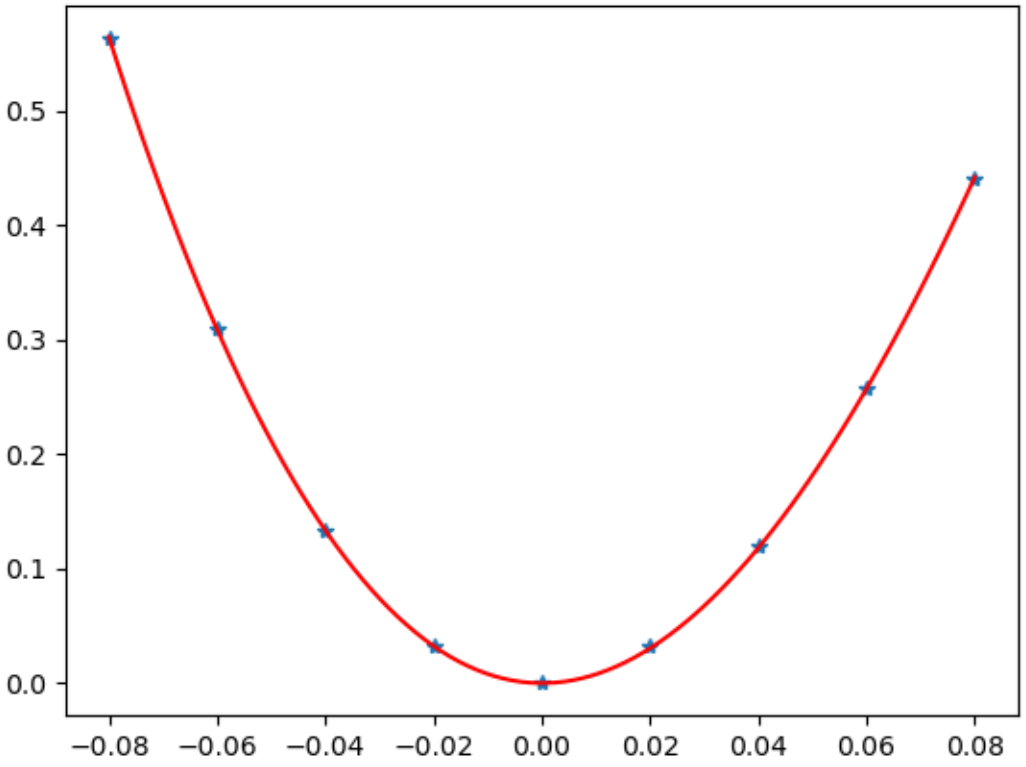
```
ELASTIC3RD version: 7.0.2
Copyright © FGMS@HIT & JUST

BaseVec:
     5.4647128853     0.0000000210     -0.0000000454
     -0.0000000000     5.4647128382     -0.0000000222
     0.0000000000     0.0000000000     5.4647129020
Lattice Parameter:
    a         b         c      alpha   beta    gamma
5.464713  5.464713  5.464713  90.000  90.000  90.000
Volume : 163.1931949715094

[11 12 44]
[153.05111127  65.0171889   73.22518077]
[111 112 123 144 155 456]
[-703.9889652  -435.4462948   -88.79485798   71.7458725  -257.35598339
  -51.04707056]
```

- Higher-order effect and figures

Run *elastic3rd post -n 4 -fig 1* to include the fourth-order term when recovering the constants and to show the fitting figures of every strain mode. The higher-order-effect treatment is documented in M. Liao, Y. Liu, Y. Wang et al., Revisiting the third-order elastic constants of diamond: the higher-order effect, Diamond Relat. Mater. 117 (2021) 108490, doi:10.1016/j.diamond.2021.108490:



- By calling the *post*, *post_single* or *post_mode* functions from *elastic3rd.post.post*

```
import elastic3rd.post.post2 as espost2
C, Cerr = espost2.post_single(x, E, StrainIn, CoefMethod='fit', V0=V0,
                              Flag_Ord=4, INPUT='INPUT.e3rd', flag_low=0)
```

where `x` are the strain amplitudes (fraction), `E` the energy densities and `StrainIn` the strain modes; `Flag_Ord` may be set up to 9.

CHAPTER

SEVEN

FAQ

Which crystal type should I set for `CrystalType`?

Use the elastic (Laue) class of the crystal, not the space group: i, c1, c2, h1, h2, r1, r2, t1, t2, o, m, n (long names such as `cubic1` also work). The 32 crystallo- graphic point groups collapse into these classes; helpers in `elastic3rd.symmetry.symdata` (`group2crytyp`, `space_group_number2CrystalType`) can map a point group or space group for you.

My VASP stress run returns nothing / wrong constants.

For the strain-stress method (`FlagSE S`) the stress must be written to `OUTCAR`: set `ISIF = 2` in the INCAR. The `INCAR` written by the VASP provider for ion relaxation already does this; a hand-written INCAR must.

What are the units?

Strain amplitudes in `MaxStrain`/`MinStrain`/`STRAINAMP` are given in percent. Energies are eV per cell (converted also to Hartree, kJ/mol and kcal/mol in the outputs), stresses GPa, and all reported elastic constants GPa.

Can I use my own strain modes?

Yes. Set `CalMode S` and provide a `STRAINMODE` file with one Voigt 6-vector (`xx yy zz yz xz xy`) per line. With `CalMode W, GenSM 1` the built-in modes are written to a `STRAINMODE` file, which you can inspect and then reuse.

Can I control the exact strain amplitudes?

Yes. Put a file named `STRAINAMP` in the working folder containing the amplitudes (in percent); it overrides `MinStrain`, `MaxStrain`, `STEPS` and `StrainType`. For `CoefMethod num` the list must be symmetric around 0 (e.g. `-5 -2.5 0 2.5 5`) with `STEPS = 5` and the desired `Acc`.

The job was interrupted. Must I start over?

No. Set `Continue 1` in `INPUT.e3rd` and run again from the same folder: existing `Energy_*.txt`/`Stress_*.txt` files are read instead of recalculated. New strain points are still computed.

My relaxed structures at each strain drift / symmetry is not kept.

Set `StableEst 1` (relax ions at fixed cell) if you want the lattice to stay exactly as deformed; relaxions with full cell freedom can drift. `AddPert 1` helps when ion relxations stall, and `Geom` handles the pre-deformation equilibrium structure.

How do I get 4th-order constants?

Set `Ord 4` (built-in mode tables and coefficient relations exist for all 11 classes at 2nd/3rd/4th order). For 5th order, only c1/c2/t1/t2/o/m/n are tabulated. You can also always recover higher-order information afterwards from stored data with `elastic3rd post -n N`.

How do I choose a different model for a machine-learning provider?

Add a `settings.yaml` in the working folder with `model`, `device`, `optimizer`, `ase_filter`, `steps` keys (the accepted model names are the ones of the corresponding upstream package, e.g. `M3GNet-MP-2021.2.8-PES` for `matgl`). Locally trained NequIP/DeepMD/AlphaNet models are read from their default cache locations or via `model_path`/`weights_path`.

Can I run Elastic3rd inside a Jupyter notebook or a batch script?

Yes, the CLI is a thin wrapper around `elastic3rd.elastic.elastic3(INPUT="...")`. Copy the pattern of `examples/runElastic3rd.py`.

CHAPTER

EIGHT

ELASTIC3RD API

The public API is function-based (there is no `Elastic3rd` class). The entry points most users need:

``elastic3rd.elastic``
: `elastic3(INPUT="INPUT.e3rd")` – the full workflow described in Get Started

``elastic3rd.esutils``
: input parsing (`read_input`), strain list generation (`gen_strain_list`), strain-mode file IO (`read_strainmode`), logging, crystal type mapping

``elastic3rd.crystal.lattice``
: lattice utilities: `print_lattice` (lattice parameters and volume)

``elastic3rd.crystal.deform``
: strain to deformation gradient (`strain2deformgrad`)

``elastic3rd.symmetry``
: symmetry coefficient tables and mode generation: `symmetry` (energy method), `symmetry_stress` (stress method), `symdata` (independent-constant relations), `gencoef` (built-in strain-mode library), `print_cijk` (label results)

``elastic3rd.energy``
: one provider module per `EnergyCode` plus the shared helpers `glue` (units, run command) and `ml_ff` (base class for the ML providers)

``elastic3rd.post``
: `post`/`post_mode`/`post_single` (refit collected data, higher-order effects), `esfit` (polynomial fits and figures), `post_nd` (general numerical-differentiation post)

``elastic3rd.tools``
: auxiliary scripts: `runElastic3rd.py` (workflow launcher to copy into a work folder), `runAll.py` (pymatgen/dfttk-based VASP input generation and batch runs), `nd_coef` (finite-difference weights), `shockwave` (shock compression analysis), `sep_nd` (component-wise SOEC/TOEC by ND), `post_asym` (convergence vs strain-window study)

``elastic3rd.scripts``
: `run_elastic3rd` – the `elastic3rd` console command

Typical programmatic use:

```
import elastic3rd.elastic as e3
e3.elastic3(INPUT="INPUT.e3rd")
```

Post-processing only, with error estimates and R^2 from the weighted fit:

```
import elastic3rd.post.post2 as espost2
C, Cerr = espost2.post_single(x, E, StrainIn, CoefMethod="fit",
                              V0=V0, Flag_Ord=3, INPUT="INPUT.e3rd")
```

Provider API (to plug in a new code) is documented in *Get Started*.

A full autodoc reference is planned (TODO).

CHAPTER

NINE

CITING

If you use Elastic3rd in your research, please cite:

- Mingqing Liao, ShunLi Shang, Yong Liu, Zhigao Chen, Xingjun Liu, Huazhou Song, Fu Xu. *Elastic3rd: A tool for calculating third-order elastic constants from first principles*, Computer Physics Communications 261 (2021) 107777.
- Mingqing Liao, et al. *Strain-stress method for calculating high-order elastic constants*, Computer Physics Communications 280 (2022) 108478.

and give the following link: Elastic3rd

CHAPTER

TEN

AUTHOR INFORMATION

Mingqing Liao
liaomq1900127@163.com
FGMS group @ Harbin Institute of Technology(HIT)
Jiangsu University of Science and Technology(JSIT)

CHAPTER

ELEVEN

INDICES AND TABLES

- genindex
- modindex
- search